

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Artificial intelligence

Practical no: 2

Aim:

- Implement the A* Search algorithm for solving a pathfinding problem.
- Implement the Recursive Best-First Search algorithm for the same problem.
- Compare the performance and effectiveness of both algorithms.

```
Command Prompt
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\IT-22>pip install matplotlib networkx
Collecting matplotlib
  Downloading matplotlib-3.11.0-cp314-cp314-win_amd64.whl.metadata (80 kB)
Collecting networkx
  Downloading networkx-3.6.1-py3-none-any.whl.metadata (6.8 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.3-cp314-cp314-win_amd64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Downloading fonttools-4.63.0-cp314-cp314-win_amd64.whl.metadata (121 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.0-cp314-cp314-win_amd64.whl.metadata (5.2 kB)
Requirement already satisfied: numpy>=1.25 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (2.5.0)
Requirement already satisfied: packaging>=20.0 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (26.2)
Collecting pillow>=9 (from matplotlib)
  Downloading pillow-12.3.0-cp314-cp314-win_amd64.whl.metadata (9.3 kB)
Collecting pyparsing>=3 (from matplotlib)
  Downloading pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: python-dateutil>=2.7 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in .\AppData\Local\Programs\Python\Python314\Lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Downloading matplotlib-3.11.0-cp314-cp314-win_amd64.whl (9.5 MB)
----- 9.5/9.5 MB 4.3 MB/s 0:00:02
Downloading networkx-3.6.1-py3-none-any.whl (2.1 MB)
----- 2.1/2.1 MB 4.1 MB/s 0:00:00
Downloading contourpy-1.3.3-cp314-cp314-win_amd64.whl (232 kB)
----- 232/232 kB 4.1 MB/s 0:00:00
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
----- 8.3/8.3 kB 4.7 MB/s 0:00:00
Downloading fonttools-4.63.0-cp314-cp314-win_amd64.whl (2.3 MB)
----- 2.3/2.3 MB 4.7 MB/s 0:00:00
Downloading kiwisolver-1.5.0-cp314-cp314-win_amd64.whl (75 kB)
----- 75/75 kB 2.3 MB/s 0:00:03
Downloading pillow-12.3.0-cp314-cp314-win_amd64.whl (7.2 MB)
----- 7.2/7.2 MB 2.3 MB/s 0:00:03
Downloading pyparsing-3.3.2-py3-none-any.whl (122 kB)
----- 122/122 kB 2.3 MB/s 0:00:00
Installing collected packages: pyparsing, pillow, networkx, kiwisolver, fonttools, cycler, contourpy, matplotlib
Successfully installed contourpy-1.3.3 cycler-0.12.1 fonttools-4.63.0 kiwisolver-1.5.0 matplotlib-3.11.0 networkx-3.6.1 pillow-12.3.0 pyparsing-3.3.2

C:\Users\IT-22>
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Artificial intelligence

```
T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

# A* Search algorithm for solving a pathfinding problem.
# Example: Powai Lake to Bandstand

import heapq
import matplotlib.pyplot as plt
import networkx as nx

# -----
# 1. DATA STRUCTURE POPULATION
# -----

# Actual Road Distance g(n) between interconnected nodes
graph = {
    'Powai Lake': {'Andheri': 7, 'Ghatkopar': 6},
    'Andheri': {'Santacruz': 5, 'Bandra': 8},
    'Ghatkopar': {'Kurla': 5},
    'Santacruz': {'Bandra': 4},
    'Kurla': {'Bandra': 7},
    'Bandra': {'Bandstand': 3},
    'Bandstand': {}
}

# Straight-line Heuristic Distance h(n) from each node to Bandstand
heuristics = {
    'Powai Lake': 18,
    'Andheri': 11,
    'Ghatkopar': 14,
    'Santacruz': 6,
    'Kurla': 9,
    'Bandra': 3,
    'Bandstand': 0
}

# Rough spatial coordinates (X, Y) for graph plotting
node_positions = {
    'Powai Lake': (4, 8),
    'Andheri': (2, 6),
    'Ghatkopar': (6, 6),
    'Santacruz': (2, 4),
    'Kurla': (6, 4),
    'Bandra': (4, 2),
    'Bandstand': (4, 0)
}

# -----
# 2. A* SEARCH ALGORITHM
# -----

def a_star_search(graph, heuristics, start, goal):
    # Priority Queue stores tuples:
    # (f_score, current_node, path_taken, cumulative_g_score)

    priority_queue = [(heuristics[start], start, [start], 0)]
    visited = set()

    while priority_queue:
        # Always extract node having minimum f(n)=g(n)+h(n)
        f_score, current, path, g_score = heapq.heappop(priority_queue)

        if current in visited:
            continue

        visited.add(current)

        # Goal Test
        if current == goal:
            return path, g_score

Ln: 231 Col: 25
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Artificial intelligence

```
T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

# -----
# 3. GRAPH PLOTTING AND VISUALIZATION
# -----

# Build NetworkX Graph
G = nx.DiGraph()

for node, neighbours in graph.items():
    for neighbour, weight in neighbours.items():
        G.add_edge(node, neighbour, weight=weight)

plt.figure(figsize=(10,8))

# Highlight optimal path
path_edges = list(zip(optimal_path, optimal_path[1:]))
normal_edges = [edge for edge in G.edges() if edge not in path_edges]

# Draw Nodes
nx.draw_networkx_nodes(
    G,
    node_positions,
    node_size=2500,
    node_color="lightblue"
)

# Draw Normal Edges
nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=normal_edges,
    width=1.5,
    edge_color="gray",
    arrows=True
)

# Draw Optimal Path
nx.draw_networkx_edges(
    G,
    node_positions,
    edgelist=path_edges,
    width=3.5,
    edge_color="darkorange",
    arrows=True
)

# Node Labels with Heuristics
node_labels = {
    node: f"{node}\nh(n)={heuristics[node]}"
    for node in G.nodes()
}

nx.draw_networkx_labels(
    G,
    node_positions,
    labels=node_labels,
    font_size=9,
    font_weight="bold"
)

# Edge Labels
edge_labels = nx.get_edge_attributes(G, "weight")
edge_labels = {k: f"{v} km" for k, v in edge_labels.items()}

nx.draw_networkx_edge_labels(
    G,
    node_positions,
    edge_labels=edge_labels,
    font_color="red"
)

plt.title(
    "A* Routing Map: Dawai Lake to Bandstand\n"
)
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Artificial intelligence

```
T071_AI_Pracno_2.py - C:/...
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
===== RESTART: C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py =====
Optimal Path Discovered:
Powai Lake -> Andheri -> Bandra -> Bandstand
Total Road Distance: 18 km

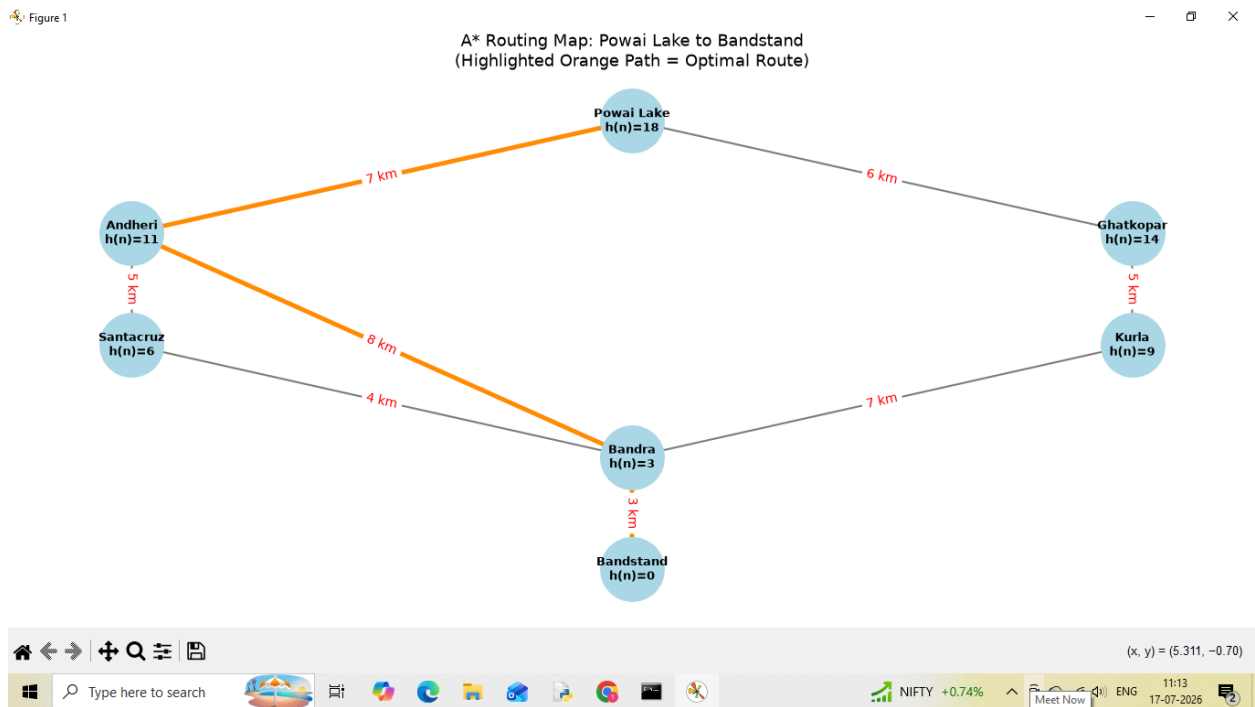
# Node Labels with h(n) values
node_labels = {}
for node in nodes:
    node_labels[node] = h(node)

nx.draw_networkx(G, node_pos=node_labels, font_size=9, font_weight='bold')

# Edge Labels with distance
edge_labels = {}
for edge in G.edges():
    edge_labels[edge] = distance(edge)

nx.draw_networkx(G, node_pos=node_labels, edge_labels=edge_labels, font_color='red')

plt.title("A* Routing Map: Powai Lake to Bandstand (Highlighted Orange Path = Optimal Route)")
plt.axis("off")
plt.tight_layout()
plt.show()
```



SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Artificial intelligence

```
*T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

# Recursive Best-First Search (RBFS)
# Example: Powai Lake to Bandstand

import matplotlib.pyplot as plt
graph = {
    'Powai Lake': {'Andheri': 7, 'Ghatkopar': 6},
    'Andheri': {'Santacruz': 5, 'Bandra': 8},
    'Ghatkopar': {'Kurla': 5},
    'Santacruz': {'Bandra': 4},
    'Kurla': {'Bandra': 7},
    'Bandra': {'Bandstand': 3},
    'Bandstand': {}
}

heuristics = {
    'Powai Lake': 18,
    'Andheri': 11,
    'Ghatkopar': 14,
    'Santacruz': 6,
    'Kurla': 9,
    'Bandra': 3,
    'Bandstand': 0
}

coords = {
    'Powai Lake': (4, 8),
    'Andheri': (2, 6),
    'Ghatkopar': (6, 6),
    'Santacruz': (2, 4),
    'Kurla': (6, 4),
    'Bandra': (4, 2),
    'Bandstand': (4, 0)
}

def rbfs_search(start, goal):

Ln: 373 Col: 0

*T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

def rbfs_search(start, goal):

    success, path, cost, _ = rbfs(
        start,
        goal,
        g=0,
        f_limit=float('inf'),
        path=[start]
    )

    return path, cost

def rbfs(node, goal, g, f_limit, path):

    if node == goal:
        return True, path, g, g

    neighbors = graph[node]

    if not neighbors:
        return False, [], 0, float('inf')

    successors = []

    for neighbor, distance in neighbors.items():

        if neighbor not in path:

            next_g = g + distance
            next_f = max(next_g + heuristics[neighbor],
                          g + heuristics[node])

            successors.append((next_f, neighbor, next_g))

    if not successors:
        return False, [], 0, float('inf')

Ln: 373 Col: 0
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Artificial intelligence

```
*T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

def rbfs_search(start, goal):
    success, path, cost, _ = rbfs(
        start,
        goal,
        g=0,
        f_limit=float('inf'),
        path=[start]
    )
    return path, cost

def rbfs(node, goal, g, f_limit, path):
    if node == goal:
        return True, path, g, g
    neighbors = graph[node]
    if not neighbors:
        return False, [], 0, float('inf')
    successors = []
    for neighbor, distance in neighbors.items():
        if neighbor not in path:
            next_g = g + distance
            next_f = max(next_g + heuristics[neighbor],
                          g + heuristics[node])
            successors.append([next_f, neighbor, next_g])
    if not successors:
        return False, [], 0, float('inf')

path, total_dist = rbfs_search('Powai Lake', 'Bandstand')
print(f"Optimal RBFS Path: (' -> '.join(path)) ({total_dist} km)")
plt.figure(figsize=(8,7))
for node, neighbors in graph.items():
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Artificial intelligence

```
*T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

for node, neighbors in graph.items():

    x1, y1 = coords[node]

    for neighbor, dist in neighbors.items():

        x2, y2 = coords[neighbor]

        is_path = (
            node in path and
            neighbor in path and
            path.index(neighbor) == path.index(node) + 1
        )

        color, width = ('#2ecc71', 3) if is_path else ('#bdc3c7', 1.5)

        plt.plot([x1, x2], [y1, y2],
                  color=color,
                  linewidth=width)

        plt.text(
            (x1+x2)/2,
            (y1+y2)/2,
            f"{dist} km",
            color='red',
            fontsize=9,
            ha='center'
        )

# Draw nodes
for node, (x, y) in coords.items():

    color = '#f1c40f' if node in path else '#3498db'

    plt.scatter(x, y, color=color, s=700)

    plt.text(
        (x1+x2)/2,
        (y1+y2)/2,
        f"{dist} km",
        color='red',
        fontsize=9,
        ha='center'
    )

# Draw nodes
for node, (x, y) in coords.items():

    color = '#f1c40f' if node in path else '#3498db'

    plt.scatter(x, y, color=color, s=700)

    plt.text(
        x,
        y,
        f"{node}\nh={heuristics[node]}",
        ha='center',
        va='center',
        color='white',
        fontsize=8,
        fontweight='bold'
    )

plt.title(
    f"RBFS Shortest Route Map\nPowai Lake to Bandstand (Total: {total_dist} km)",
    fontsize=12,
    fontweight='bold'
)

plt.axis('off')
plt.show()

Ln: 504 Col: 35
```

```
*T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

        plt.text(
            (x1+x2)/2,
            (y1+y2)/2,
            f"{dist} km",
            color='red',
            fontsize=9,
            ha='center'
        )

# Draw nodes
for node, (x, y) in coords.items():

    color = '#f1c40f' if node in path else '#3498db'

    plt.scatter(x, y, color=color, s=700)

    plt.text(
        x,
        y,
        f"{node}\nh={heuristics[node]}",
        ha='center',
        va='center',
        color='white',
        fontsize=8,
        fontweight='bold'
    )

plt.title(
    f"RBFS Shortest Route Map\nPowai Lake to Bandstand (Total: {total_dist} km)",
    fontsize=12,
    fontweight='bold'
)

plt.axis('off')
plt.show()

Ln: 525 Col: 1
```

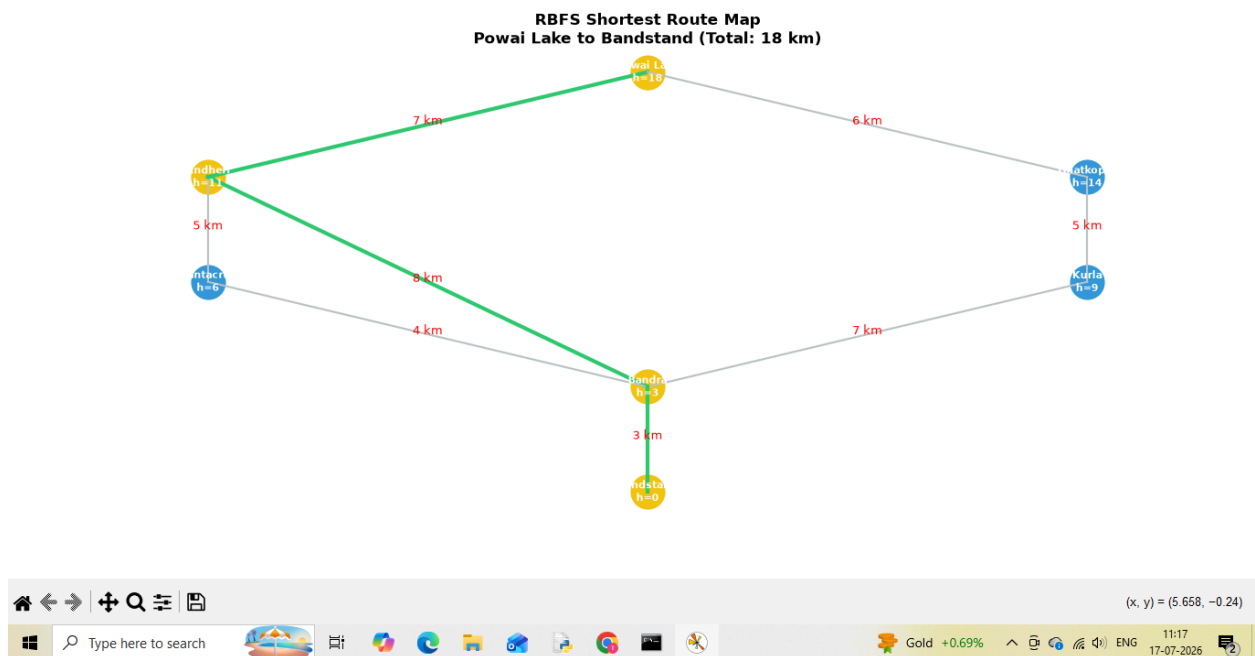
SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Artificial intelligence

```
T071_AI_Pracno_2.py
File Edit Format Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py =====
Optimal RBFS Path: Powai Lake -> Andheri -> Bandra -> Bandstand (18 km)
>>>

plt.title('')
plt.xlabel('')
plt.ylabel('')
plt.axis('off')
plt.show()
```

Figure 1



SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Artificial intelligence

```
T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

##
dict_hn = {
    'Powai Lake': 18,
    'Andheri': 11,
    'Ghatkopar': 14,
    'Santacruz': 6,
    'Kurla': 9,
    'Bandra': 3,
    'Bandstand': 0
}

dict_gn = dict(
    **{
        'Powai Lake': dict(Andheri=7, Ghatkopar=6),
        'Andheri': dict(Powai Lake=7, Santacruz=5, Bandra=8),
        'Ghatkopar': dict(Powai Lake=6, Kurla=5),
        'Santacruz': dict(Andheri=5, Bandra=4),
        'Kurla': dict(Ghatkopar=5, Bandra=7),
        'Bandra': dict(Andheri=8, Santacruz=4, Kurla=7, Bandstand=3),
        'Bandstand': dict()
    }
)

# Rename key to match dictionary
dict_gn['Andheri']['Powai Lake'] = dict_gn['Andheri'].pop('Powai Lake')
dict_gn['Ghatkopar']['Powai Lake'] = dict_gn['Ghatkopar'].pop('Powai Lake')

import queue as Q

start = 'Powai Lake'
goal = 'Bandstand'
result = ''

def get_fn(citystr):
    cities = citystr.split(" , ")
    hn = gn = 0

    for ctr in range(len(cities)-1):
        gn += dict_gn[cities[ctr]][cities[ctr+1]]

    hn = dict_hn[cities[-1]]
    return hn + gn

def expand(cityq):
    global result

    tot, citystr, thiscity = cityq.get()

    if thiscity == goal:
        result = citystr + " : : " + str(tot)
        return

    for cty in dict_gn[thiscity]:
        cityq.put((get_fn(citystr + " , " + cty),
                  citystr + " , " + cty,
                  cty))

    expand(cityq)

def main():
    cityq = Q.PriorityQueue()

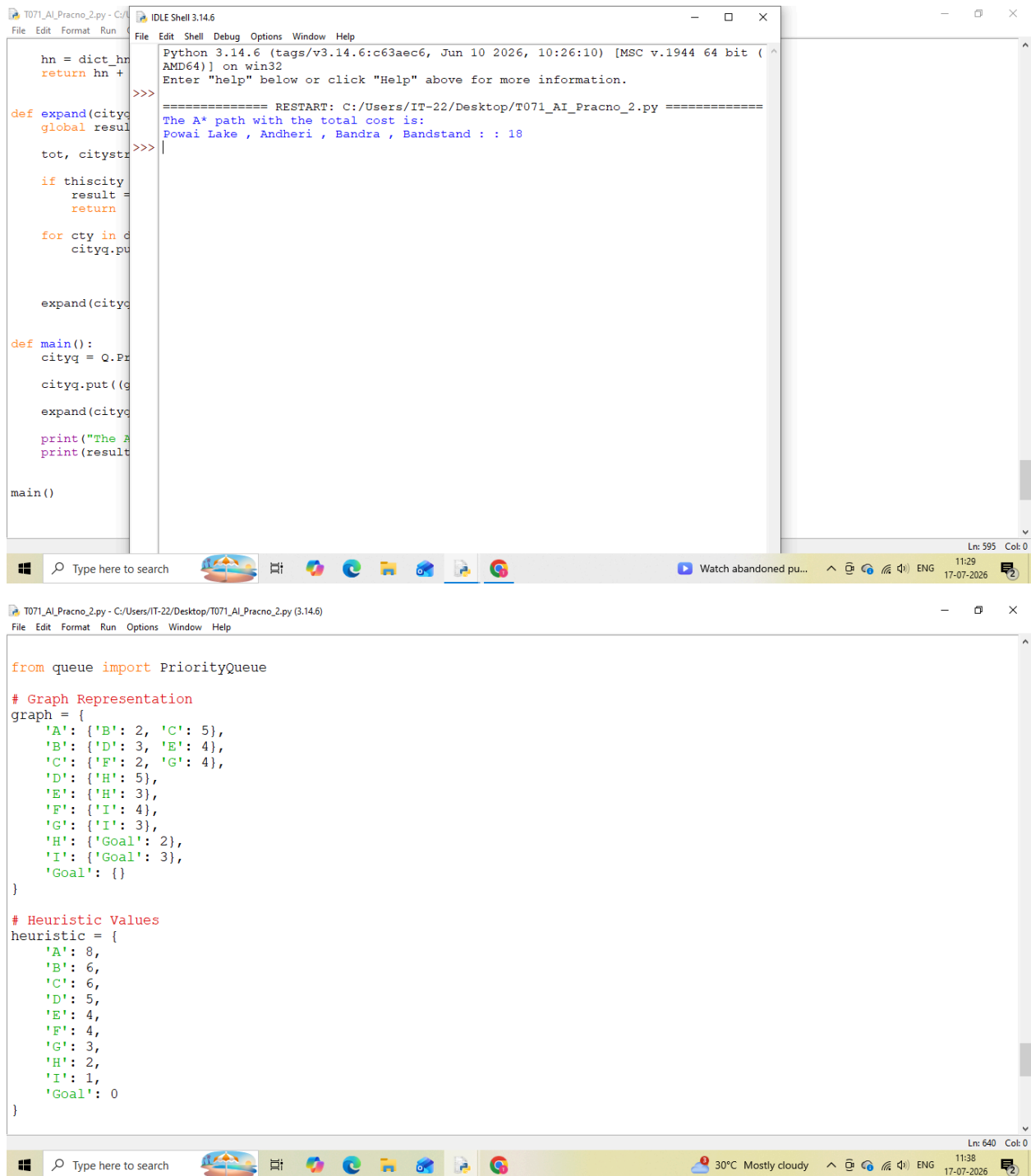
    cityq.put((get_fn(start), start, start))

    expand(cityq)

    print("The A* path with the total cost is:")
    print(result)
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Artificial intelligence



```
hn = dict_hn
return hn +

def expand(cityq):
    global result
    tot, citystr = cityq.get()

    if thiscity in result:
        return

    for cty in d:
        cityq.put((cty, tot + d[cty]))

    expand(cityq)

def main():
    cityq = Q.PriorityQueue()
    cityq.put((A, 0))
    expand(cityq)
    print("The A* path is: ")
    print(result)

main()
```

Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1914 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>> ===== RESTART: C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py =====
The A* path with the total cost is:
Powai Lake , Andheri , Bandra , Bandstand : : 18
>>>

```
from queue import PriorityQueue

# Graph Representation
graph = {
    'A': {'B': 2, 'C': 5},
    'B': {'D': 3, 'E': 4},
    'C': {'F': 2, 'G': 4},
    'D': {'H': 5},
    'E': {'H': 3},
    'F': {'I': 4},
    'G': {'I': 3},
    'H': {'Goal': 2},
    'I': {'Goal': 3},
    'Goal': {}
}

# Heuristic Values
heuristic = {
    'A': 8,
    'B': 6,
    'C': 6,
    'D': 5,
    'E': 4,
    'F': 4,
    'G': 3,
    'H': 2,
    'I': 1,
    'Goal': 0
}
```

A* program

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Artificial intelligence

```
T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

/

start = 'A'
goal = 'Goal'
|
# Priority Queue stores (f, g, path)
pq = PriorityQueue()
pq.put((heuristic[start], 0, [start]))

visited = set()

while not pq.empty():

    f, g, path = pq.get()
    node = path[-1]

    if node == goal:
        print("\nBest Path :", " -> ".join(path))
        print("Total Cost =", g)
        break

    if node in visited:
        continue

    visited.add(node)

    for neighbour, cost in graph[node].items():

        new_g = g + cost
        new_f = new_g + heuristic[neighbour]

        print(

if node == goal:
    print("\nBest Path :", " -> ".join(path))
    print("Total Cost =", g)
    break

if node in visited:
    continue

visited.add(node)

for neighbour, cost in graph[node].items():

    new_g = g + cost
    new_f = new_g + heuristic[neighbour]

    print(
        path + [neighbour],
        "g =", new_g,
        "h =", heuristic[neighbour],
        "f =", new_f
    )

    pq.put((new_f, new_g, path + [neighbour]))

Ln: 640 Col: 0
```

```
T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

if node == goal:
    print("\nBest Path :", " -> ".join(path))
    print("Total Cost =", g)
    break

if node in visited:
    continue

visited.add(node)

for neighbour, cost in graph[node].items():

    new_g = g + cost
    new_f = new_g + heuristic[neighbour]

    print(
        path + [neighbour],
        "g =", new_g,
        "h =", heuristic[neighbour],
        "f =", new_f
    )

    pq.put((new_f, new_g, path + [neighbour]))

Ln: 640 Col: 0
```

SHETH L.U.J AND SIR M.V COLLEGE

SUBJECT:- Artificial intelligence

```
T071_AI_Pracno_2.py - C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py (3.14.6)
File Edit Format Run Options Window Help

)

# Node Labels with Heuristics
node_labels = {
    node: f"{node}\nh(n)={heuristics[node]}"
    for node in G.nodes()
}

nx.draw_networkx_labels(
    G,
    node_positions,
    labels=node_labels,
    font_size=9,
    font_weight="bold"
)

# Edge Labels
edge_labels = nx.get_edge_attributes(G, "weight")
edge_labels = {k: f"{v} km" for k, v in edge_labels.items()}

nx.draw_networkx_edge_labels(
    G,
    node_positions,
    edge_labels=edge_labels,
    font_color="red"
)

plt.title(
    "A* Routing Map: Powai Lake to Bandstand\n"
    "(Highlighted Orange Path = Optimal Route)",
    fontsize=13
)

plt.axis("off")
plt.tight_layout()
plt.show()

Ln: 356 Col: 15

Type here to search NIFTY +0.74% 11:12 17-07-2026

IDLE Shell 3.14.6
File Edit Shell Debug Options Window Help
Python 3.14.6 (tags/v3.14.6:c63a6c6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/IT-22/Desktop/T071_AI_Pracno_2.py =====
['A', 'B'] g = 2 h = 6 f = 8
['A', 'C'] g = 5 h = 6 f = 11
['A', 'B', 'D'] g = 5 h = 5 f = 10
['A', 'B', 'E'] g = 6 h = 4 f = 10
['A', 'B', 'D', 'H'] g = 10 h = 2 f = 12
['A', 'B', 'E', 'H'] g = 9 h = 2 f = 11
['A', 'C', 'F'] g = 7 h = 4 f = 11
['A', 'C', 'G'] g = 9 h = 3 f = 12
['A', 'C', 'F', 'I'] g = 11 h = 1 f = 12
['A', 'B', 'E', 'H', 'Goal'] g = 11 h = 0 f = 11

Best Path : A -> B -> E -> H -> Goal
Total Cost = 11
>>>
```